

Machine Learning Lab Codes

Lab 1 — K-Means Clustering

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

iris = load_iris()
X = iris.data

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)

labels = kmeans.labels_

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title('K-means Clustering on Iris Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Lab 2 — Hierarchical Clustering

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

iris = load_iris()
X = iris.data

agg_cluster = AgglomerativeClustering(n_clusters=3)
agg_labels = agg_cluster.fit_predict(X)

linkage_matrix = linkage(X, method='ward')

dendrogram(linkage_matrix)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Sample Index')
plt.ylabel('Distance')
plt.show()

plt.scatter(X[:, 0], X[:, 1], c=agg_labels, cmap='viridis')
plt.title('Hierarchical Clustering on Iris Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Lab 3 — Apriori Algorithm

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

dataset = [
    ['Milk', 'Bread', 'Eggs'],
    ['Milk', 'Apple', 'Eggs', 'Cheese'],
    ['Bread', 'Apple', 'Cheese'],
    ['Bread', 'Eggs'],
    ['Milk', 'Bread', 'Apple', 'Cheese']
]

te = TransactionEncoder()
```

```

te_ary = te.fit(dataset).transform(dataset)

df = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df, min_support=0.2, use_colnames=True)

rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.7
)

print("Frequent Itemsets:")
print(frequent_itemsets)

print("\nAssociation Rules:")
print(rules)

```

Lab 4 — Market Basket Analysis

```

import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

dataset = [
    ['Coffee', 'Bread', 'Butter'],
    ['Tea', 'Coffee', 'Sugar', 'Bread'],
    ['Milk', 'Sugar', 'Butter'],
    ['Tea', 'Coffee', 'Milk'],
    ['Tea', 'Bread', 'Butter']
]

te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)

df = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df, min_support=0.2, use_colnames=True)

rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.7
)

print("Frequent Itemsets:")
print(frequent_itemsets)

print("\nAssociation Rules:")
print(rules)

```

Lab 5 — Reinforcement Learning

```

import numpy as np

alpha = 0.1
gamma = 0.9

num_states = 5
num_actions = 3

Q = np.zeros((num_states, num_actions))

current_state = 0
action = 1
next_state = 2
reward = 1

Q[current_state, action] += alpha * (
    reward +
    gamma * np.max(Q[next_state, :]) -

```

```

    Q[current_state, action]
)

optimal_action = np.argmax(Q[current_state, :])

print("Q Table:")
print(Q)

print("Optimal Action:")
print(optimal_action)

```

Lab 6 — Time Series Analysis

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA

np.random.seed(42)

time = np.arange(100)
data = np.random.randn(100) + 0.1 * time

plt.plot(time, data)
plt.title('Non-Stationary Time Series')
plt.xlabel('Time')
plt.ylabel('Value')
plt.show()

def check_stationarity(ts):
    result = adfuller(ts)

    print('ADF Statistic:', result[0])
    print('p-value:', result[1])
    print('Critical Values:', result[4])

stationary_data = np.diff(data, n=1)

print("\nDickey-Fuller Test for Stationarity:")
check_stationarity(stationary_data)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

plot_acf(stationary_data, ax=ax1, lags=20)
plot_pacf(stationary_data, ax=ax2, lags=20)

plt.show()

order = (1, 1, 1)

model = ARIMA(data, order=order)
results = model.fit()

forecast_steps = 10

forecast_values = results.get_forecast(
    steps=forecast_steps
).predicted_mean

plt.plot(time, data, label='Original Data')

plt.plot(
    np.arange(100, 100 + forecast_steps),
    forecast_values,
    label='Forecast'
)

plt.title('Time Series Forecasting with ARIMA')
plt.xlabel('Time')
plt.ylabel('Value')
plt.legend()
plt.show()

```

Lab 7 — Cross Validation and AdaBoost

```
import numpy as np

from sklearn.model_selection import cross_val_score, KFold
from sklearn.ensemble import RandomForestClassifier

X, y = ...

rf_classifier = RandomForestClassifier()

kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

cross_val_scores = cross_val_score(
    rf_classifier,
    X,
    y,
    cv=kf
)

print("Cross-Validation Scores:", cross_val_scores)

print(
    "Mean Cross-Validation Score:",
    np.mean(cross_val_scores)
)

from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

base_classifier = DecisionTreeClassifier(max_depth=1)

adaboost_classifier = AdaBoostClassifier(
    base_classifier,
    n_estimators=50,
    random_state=42
)

adaboost_classifier.fit(X_train, y_train)

y_pred = adaboost_classifier.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```